



北京交通大学
BEIJING JIAOTONG UNIVERSITY

《数据库系统原理》课程设计

计算机学院人工智能专业

张鲈洋 22281052 AI2201 班

日期：2025 年 6 月 12 日

北京交通大学计算机科学与技术学院

目录

1	实验概述	1
1.1	实验背景与目的	1
1.2	实验原理与关键技术	1
1.2.1	存储系统分层架构	1
1.2.2	磁盘存储管理 (DiskManager)	2
1.2.3	缓冲池管理 (BufferPoolManager)	3
1.2.4	记录管理 (RecordManager)	4
2	任务一：缓冲池管理器实现	7
2.1	实验过程与实现细节	7
2.1.1	任务 1.1: 磁盘存储管理器 (DiskManager)	7
2.1.2	任务 1.2: LRU 替换策略 (LRUReplacer)	9
2.1.3	任务 1.3: 缓冲池管理器 (BufferPoolManager)	11
2.2	任务一遇到的问题及解决方案	13
2.2.1	DiskManager 的接口契约问题	13
2.2.2	BufferPoolManager 与 Page 类的接口适配问题	15
3	任务二：记录管理器实现	17
3.1	记录管理器的形象化理解	17
3.1.1	管理员的核心工作流程	18
3.2	实验过程与实现细节	19
3.2.1	任务 2.1: 记录操作 (RMFileHandle)	19
3.2.2	任务 2.2: 记录迭代器 (RMScan)	20
3.3	任务二遇到的问题及解决方案	21
3.3.1	核心问题: SimpleTest 中因状态不一致导致的越界读取	21
4	实验总结与心得体会	23
A	附录：实验代码	25
A.1	disk_manager.cpp	25
A.2	lru_replacer.cpp	30
A.3	buffer_pool_manager.cpp	31

A.4	rm_file_handle.cpp	36
A.5	rm_scan.cpp	40

1 实验概述

1.1 实验背景与目的

数据库管理系统 (DBMS) 是现代软件系统的核心基础设施，而存储引擎则是 DBMS 的基石，它负责管理数据在易失性内存与持久化磁盘之间的流动与组织。存储引擎的性能和可靠性直接决定了整个数据库系统的上限。

本次实验的核心目的在于通过从零开始，亲手设计并实现一个名为 Rucbase 的小型数据库系统的底层存储管理器。通过这个过程，期望能够深入理解和掌握以下关键知识点：

- 现代数据库系统中基于“页”的内外存数据交换机制。
- 缓冲池 (Buffer Pool) 作为磁盘 I/O 性能瓶颈核心解决方案的设计与实现。
- 经典的页面替换算法——最近最少使用 (LRU) 策略的原理与高效实现。
- 在页面之上，如何组织和管理记录 (Record)，包括元数据、空闲空间等的维护。
- 学习在严格的接口和约束下进行系统级编程，并通过单元测试验证模块的正确性与健壮性。

最终目标是为 Rucbase 上层模块（如执行器、事务管理等）提供一个可靠、高效的数据存取接口。

1.2 实验原理与关键技术

本次实验的实现围绕着存储系统的几个核心概念展开，它们共同构成了一个分层、协作的体系结构。

1.2.1 存储系统分层架构

数据库的存储系统并非铁板一块，而是遵循分层设计的思想，以实现各模块的解耦和高内聚。一个典型的存储引擎可以看作是执行器与操作系统文件系统之间的桥梁。其内部层次结构大致如下：

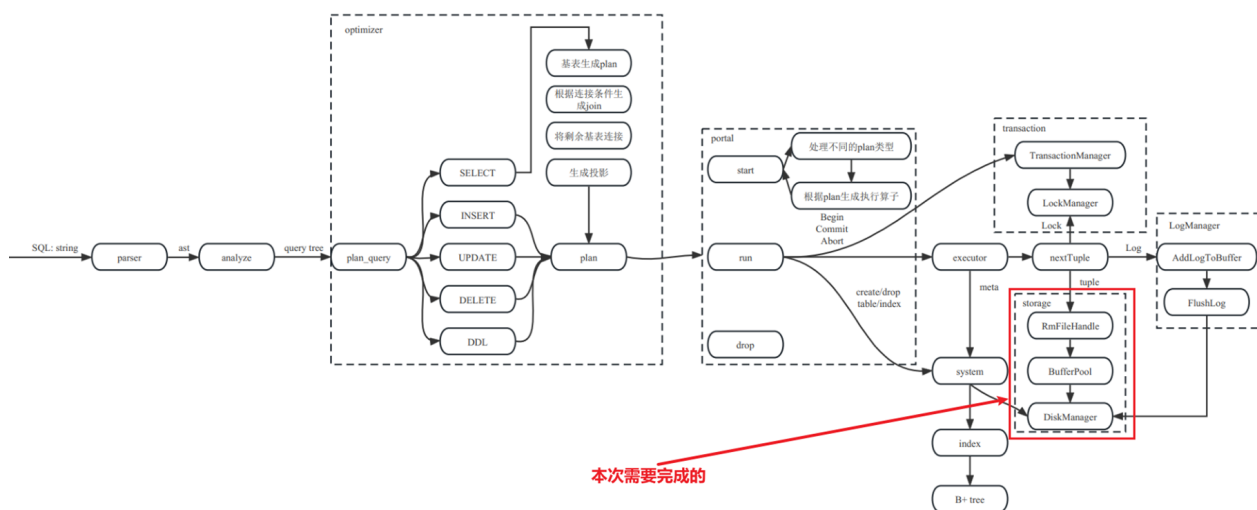


图 1: 存储系统在数据库中的位置及内部分层

- **记录管理层 (Record Layer):** 面向执行器的最高层，负责将“记录”这一逻辑概念映射到物理存储上。本实验中的 `RmFileHandle` 和 `RmScan` 即属于此层。
- **缓冲池管理层 (Buffer Pool Layer):** 核心的内外存调度层，负责管理内存中的页面缓存，以减少昂贵的磁盘访问。本实验中的 `BufferPoolManager` 和 `LRUReplacer` 构成了这一层。
- **磁盘管理层 (Disk Layer):** 最底层的抽象，负责将对“页面”的请求转换为对具体文件具体偏移量的字节读写操作。本实验中的 `DiskManager` 属于此层。

本次实验正是按照从底层到上层的顺序 (Disk -> Buffer Pool -> Record) 来构建这个存储引擎。

1.2.2 磁盘存储管理 (DiskManager)

`DiskManager` 是存储引擎与操作系统文件 I/O 之间的“翻译官”。它将上层模块对逻辑页面 (Page) 的读写请求，转换为对物理文件字节流的操作。其核心思想是：

1. **页面抽象:** 将一个磁盘文件视为一个由连续的、定长的页面 (Page) 组成的数组。每个页面拥有一个从 0 开始的唯一页面号 (`page_no`)。
2. **寻址方式:** 对一个特定页面的访问，通过公式 $\text{offset} = \text{page_no} * \text{PAGE_SIZE}$ 计算出其在文件中的字节偏移量。然后利用 Linux 的 `lseek()` 系统调用将文件指针移动到该位置。
3. **读写操作:** 定位到正确偏移量后，使用 `read()` 和 `write()` 系统调用来读写恰好一个页面大小 (`PAGE_SIZE`) 的数据。

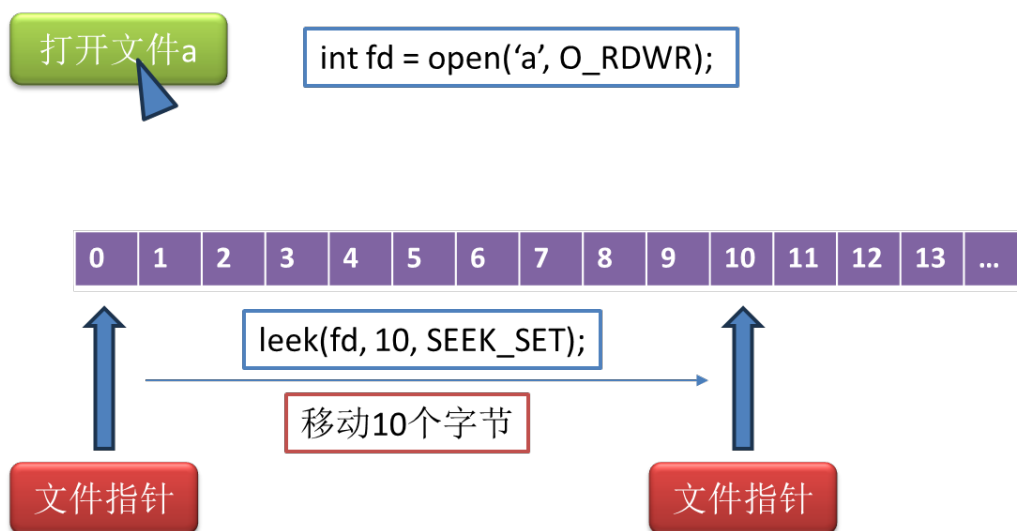


图 2: DiskManager 运行原理

此外，DiskManager 还封装了文件的创建、打开、关闭和删除等基本操作，为上层模块屏蔽了底层文件系统的复杂细节。其核心接口定义如下：

```

1 class DiskManager {
2 public:
3     // ...
4     // 读写页面
5     void write_page(int fd, page_id_t page_no, const char *offset, int num_bytes);
6     void read_page(int fd, page_id_t page_no, char *offset, int num_bytes);
7     // 对指定文件分配页面编号
8     page_id_t allocate_page(int fd);
9     // 文件操作
10    void create_file(const std::string &path);
11    int open_file(const std::string &path);
12    void close_file(int fd);
13    void destroy_file(const std::string &path);
14 };

```

Listing 1: DiskManager 核心接口

1.2.3 缓冲池管理 (BufferPoolManager)

直接访问磁盘的延迟极高，是数据库性能的主要瓶颈。缓冲池（Buffer Pool）是解决这一问题的核心技术。它在内存中开辟一片区域，用于缓存从磁盘读出的页面，使得后续对同一页面的访问可以直接在内存中完成，从而极大地提升性能。

核心数据结构 一个高效的缓冲池管理器通常由以下几个核心数据结构组成：

- `pages_`: 一个 Page 对象数组，代表物理的缓冲池内存。数组的每个元素称为一个“帧”（Frame），其大小与磁盘页面大小一致。

- `page_table_`: 一个哈希表（在本项目中为 `std::unordered_map`），用于建立从逻辑页面 ID (PageId) 到其在 `pages_` 数组中物理帧号 (`frame_id_t`) 的快速映射。这使得我们能够以 $O(1)$ 的时间复杂度判断一个页面是否在缓冲池中。
- `free_list_`: 一个链表（在本项目中为 `std::list`），用于存放当前完全空闲的帧的编号。当需要加载新页面时，优先从该链表中获取空闲帧。
- `replacer_`: 一个指向替换策略对象的指针。当 `free_list_` 为空时，由它来决定应该淘汰哪个“有主”的页面以腾出空间。

LRU 页面替换策略 当缓冲池已满且没有空闲帧时，必须选择一个当前已缓存的页面进行“淘汰” (Evict)。LRU (Least Recently Used, 最近最少使用) 是一种常用且高效的策略。其核心思想是：一个页面如果在过去的一段时间内最久没有被访问，那么它在未来被再次访问的概率也最低。

为了高效实现 LRU，本项目采用了经典的“双向链表 + 哈希表”的组合：

- `std::list<frame_id_t> LRUList_`: 一个双向链表，用于维护所有“可被淘汰”的页面的访问顺序。链表头部代表“最近”被访问（或 `unpin`）的页面，而链表尾部代表“最久”未被访问的页面。
- `std::unordered_map<frame_id_t, ...> LRUHash_`: 一个哈希表，键为帧号，值为该帧号在链表中的迭代器（指针）。

这种设计使得 `pin`（从 LRU 中移除一个节点）、`unpin`（将一个节点移到链表头）和 `victim`（从链表尾部移除一个节点）这三个核心操作的时间复杂度都能达到 $O(1)$ 。



图 3: LRU 替换策略的数据结构

1.2.4 记录管理 (RecordManager)

记录管理器建立在缓冲池之上，负责在页面内部组织和管理逻辑记录。本次实验采用的是定长记录的堆文件 (Heap File) 组织方式。

文件与页面组织结构 数据的物理组织结构是记录管理的基础，参考 `page.h` 和实验指导 PPT，其结构如下：

- **文件头 (RmFileHdr):** 存储在文件的第 0 页 (`RM_FILE_HDR_PAGE`)，它是一个全局的元数据结构，定义了整个文件的属性，如：
 - `record_size`: 文件中每条记录的固定大小。
 - `num_pages`: 文件已分配的总页数。
 - `num_records_per_page`: 根据 `record_size` 计算出的每个数据页能容纳的最大记录数。
 - `first_free_page_no`: 指向一个链表的头部，该链表串联了所有内部有空闲槽位的页面。这是实现高效插入的关键。
 - `bitmap_size`: 每个数据页中位图区域的大小。
- **数据页结构:** 从第 1 页开始的每个数据页，其内部结构被划分为三个部分：
 1. **页面头 (RmPageHdr):** 存储该页的元数据，包括 `num_records` (当前已存储记录数) 和 `next_free_page_no` (在空闲页链表中指向下一个空闲页)。
 2. **位图 (Bitmap):** 紧跟页面头的一段连续内存。位图中的每一个比特位 (bit) 对应一个记录槽位 (slot)。若比特位为 1，表示该槽位已存有记录；若为 0，表示该槽位空闲。
 3. **数据槽 (Slots):** 页面中余下的主要空间，被划分为多个定长的槽位，用于实际存储记录数据。

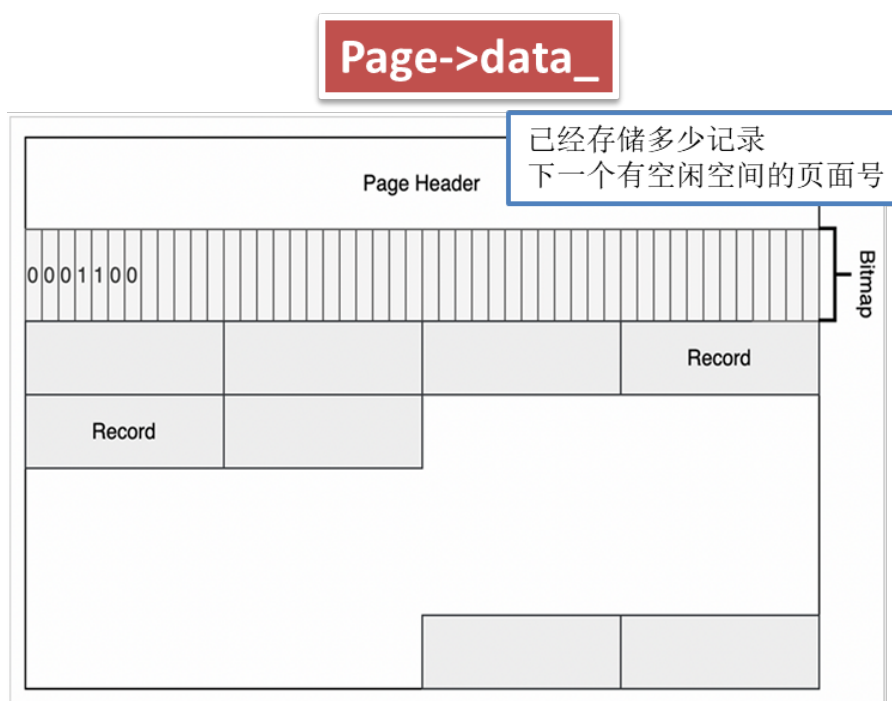


图 4: 数据页内部结构示意图

空闲空间管理 为了快速找到可插入新记录的位置,系统维护了一个由文件头 `first_free_page_no` 开始的空闲页链表。

- **插入记录时:** `RMFileHandle` 首先检查 `first_free_page_no`。若有效,则直接获取该页面进行插入。若该页面插入后变满,则 `first_free_page_no` 会更新为该页页面头中的 `next_free_page_no`,相当从链表中移除了这个已满的页面。`first_free_page_no` 本身如果无效,则需要创建新页。
- **删除记录时:** 如果一个页面因为删除操作,从“满”的状态变为“非满”,它就需要被重新加入到空闲链表的头部。具体操作是: 将其页面头中的 `next_free_page_no` 指向旧的 `first_free_page_no`, 然后更新文件头的 `first_free_page_no` 指向这个刚刚变为空闲的页面。

通过这种方式,系统避免了在插入时需要遍历整个文件的低效行为。

2 任务一：缓冲池管理器实现

任务一的目标是构建数据库存储引擎的核心——缓冲池管理器。它负责协调磁盘与内存之间的数据页交换，是后续所有数据操作的基础。此任务被分解为三个紧密相连的子任务，我按照从底层到上层的顺序逐一实现并进行了测试。

2.1 实验过程与实现细节

2.1.1 任务 1.1: 磁盘存储管理器 (DiskManager)

实现描述 DiskManager 是存储引擎的最底层，它直接与操作系统文件系统交互，提供了一个面向“页”的抽象接口。我的实现严格遵循了实验文档的要求。

- **页面读写:** write_page 和 read_page 函数是核心。它们通过公式 $\text{offset} = \text{page_no} * \text{PAGE_SIZE}$ 计算出目标页面在文件中的字节偏移量，然后调用 Linux 的 lseek() 系统调用精确定位。定位后，使用 write() 或 read() 函数传输恰好 PAGE_SIZE 大小的数据。为了保证操作的原子性和正确性，我检查了系统调用的返回值，若实际读写的字节数与预期不符，则抛出异常。

```
1 void DiskManager::write_page(int fd, page_id_t page_no, const char *offset, int
   num_bytes) {
2     off_t disk_offset = static_cast<off_t>(page_no) * PAGE_SIZE;
3     if (lseek(fd, disk_offset, SEEK_SET) != disk_offset) {
4         throw UnixError();
5     }
6     ssize_t bytes_written = write(fd, offset, num_bytes);
7     if (bytes_written != num_bytes) {
8         throw InternalError("DiskManager::write_page_Error");
9     }
10 }
11 void DiskManager::read_page(int fd, page_id_t page_no, char *offset, int
   num_bytes) {
12     off_t disk_offset = static_cast<off_t>(page_no) * PAGE_SIZE;
13     if (lseek(fd, disk_offset, SEEK_SET) != disk_offset) {
14         throw UnixError();
15     }
16     ssize_t bytes_read = read(fd, offset, num_bytes);
17     if (bytes_read != num_bytes) {
18         throw InternalError("DiskManager::read_page_Error");
19     }
20 }
```

Listing 2: DiskManager 页面读写核心逻辑

- **文件操作:** 实现了 create_file, open_file, close_file, 和 destroy_file 等一系列文件管理函数。这些函数内部封装了对应的 open(), close(), unlink() 等系统调

用。同时，为了管理文件句柄（fd）与文件路径的映射关系，并防止文件被重复打开或操作未关闭的文件，我使用了两个哈希表 path2fd_ 和 fd2path_ 来进行跟踪。

```
1 void DiskManager::create_file(const std::string &path) {
2     if (is_file(path)) {
3         throw FileExistsError(path);
4     }
5     int fd = open(path.c_str(), O_CREAT, 0666);
6     if (fd < 0) {
7         throw UnixError();
8     }
9     close(fd);
10 }
11
12 void DiskManager::destroy_file(const std::string &path) {
13     if (path2fd_.count(path)) {
14         throw FileNotClosedError(path);
15     }
16     if (!is_file(path)) {
17         throw FileNotFoundError(path);
18     }
19     if (unlink(path.c_str()) != 0) {
20         throw UnixError();
21     }
22 }
23
24 int DiskManager::open_file(const std::string &path) {
25     if (path2fd_.count(path)) {
26         throw FileNotClosedError(path);
27     }
28     if (!is_file(path)) {
29         throw FileNotFoundError(path);
30     }
31     int fd = open(path.c_str(), O_RDWR);
32     if (fd < 0) {
33         throw UnixError();
34     }
35     path2fd_[path] = fd;
36     fd2path_[fd] = path;
37     return fd;
38 }
39
40 void DiskManager::close_file(int fd) {
41     if (!fd2path_.count(fd)) {
42         throw FileNotOpenError(fd);
43     }
44     std::string path = fd2path_[fd];
45     if (close(fd) != 0) {
46         throw UnixError();
47     }
48     path2fd_.erase(path);
49     fd2path_.erase(fd);
50 }
```

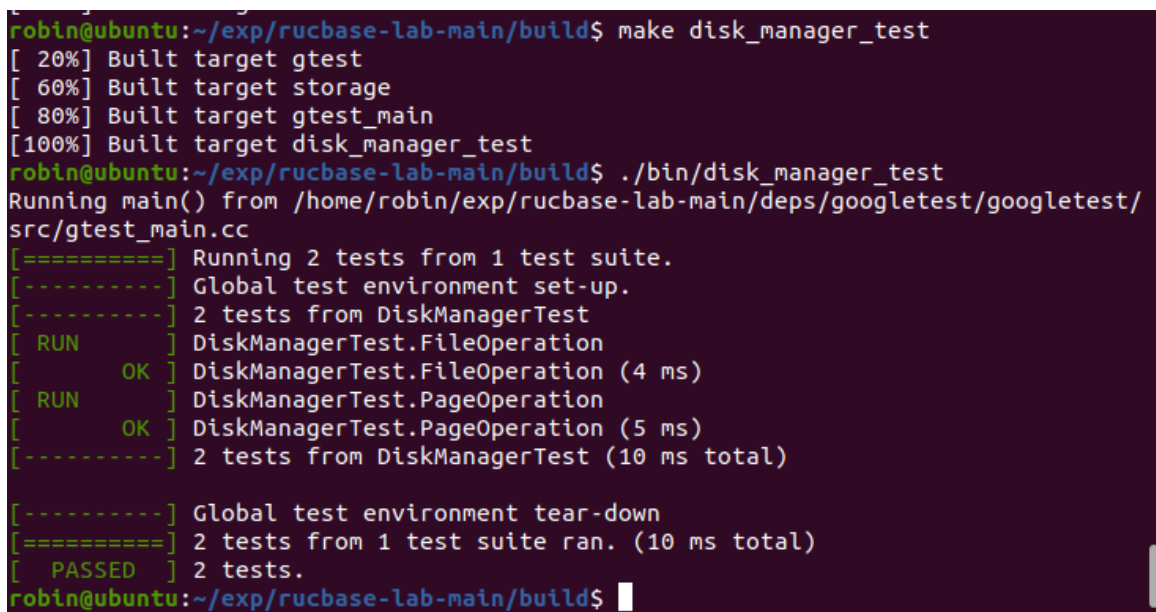
Listing 3: DiskManager 页面读写文件操作

- **页面分配:** `allocate_page` 采用简单的原子增量策略,通过 `std::atomic<page_id_t>` 类型的 `fd2pageno_` 数组为每个打开的文件维护一个页面计数器,保证了多线程环境下的页面号分配安全。

测试结果 在完成编码后,我进入 `build` 目录,执行了 `cmake` 命令 (代码4)。经过调试,最终所有测试用例均成功通过。

```
1 make disk_manager_test
2 ./bin/disk_manager_test
```

Listing 4: DiskManager cmake



```
robin@ubuntu:~/exp/rucbase-lab-main/build$ make disk_manager_test
[ 20%] Built target gtest
[ 60%] Built target storage
[ 80%] Built target gtest_main
[100%] Built target disk_manager_test
robin@ubuntu:~/exp/rucbase-lab-main/build$ ./bin/disk_manager_test
Running main() from /home/robin/exp/rucbase-lab-main/deps/googletest/googletest/src/gtest_main.cc
[=====] Running 2 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 2 tests from DiskManagerTest
[ RUN      ] DiskManagerTest.FileOperation
[ OK       ] DiskManagerTest.FileOperation (4 ms)
[ RUN      ] DiskManagerTest.PageOperation
[ OK       ] DiskManagerTest.PageOperation (5 ms)
[-----] 2 tests from DiskManagerTest (10 ms total)

[-----] Global test environment tear-down
[=====] 2 tests from 1 test suite ran. (10 ms total)
[ PASSED  ] 2 tests.
robin@ubuntu:~/exp/rucbase-lab-main/build$
```

图 5: DiskManager 单元测试通过截图

2.1.2 任务 1.2: LRU 替换策略 (LRUReplacer)

实现描述 `LRUReplacer` 负责在缓冲池满时,根据最近最少使用 (LRU) 原则,选择一个页面进行淘汰。为了实现高效的 LRU 操作,我采用了经典的“哈希表 + 双向链表”设计。

- **数据结构:** 使用 `std::list<frame_id_t>` `LRUlist_` 来维护可被淘汰页面的访问顺序。新“解钉” (unpin) 的页面被添加到链表头部,代表“最近”被使用。需要淘汰页面时,从链表尾部取出,即为“最久”未被使用的页面。同时,使用 `std::unordered_map<frame_id_t, std::list<...>::iterator>` `LRUhash_` 来存储每个帧号及其在链表中的位置 (迭代器),这使得在 `pin` 操作时,能够以 $O(1)$ 的时间复杂度快速定位并从链表中删除指定的帧。

- **核心操作:** `victim()` 从 `LRUlist_` 的尾部弹出一个帧; `pin()` 使用 `LRUhash_` 快速找到并从链表中移除一个帧; `unpin()` 将一个帧添加到 `LRUlist_` 的头部。

```
1 bool LRUReplacer::victim(frame_id_t* frame_id) {
2     std::scoped_lock lock{latch_};
3     if (LRUlist_.empty()) {
4         return false;
5     }
6     // 在我的实现中, 尾部为最久未使用
7     *frame_id = LRUlist_.back();
8     LRUhash_.erase(*frame_id);
9     LRUlist_.pop_back();
10    return true;
11 }
12
13 void LRUReplacer::pin(frame_id_t frame_id) {
14     std::scoped_lock lock{latch_};
15     auto it = LRUhash_.find(frame_id);
16     if (it != LRUhash_.end()) {
17         LRUlist_.erase(it->second);
18         LRUhash_.erase(it);
19     }
20 }
21
22 void LRUReplacer::unpin(frame_id_t frame_id) {
23     std::scoped_lock lock{latch_};
24     if (LRUhash_.count(frame_id)) {
25         return;
26     }
27     // 头部为最近使用
28     LRUlist_.push_front(frame_id);
29     LRUhash_[frame_id] = LRUlist_.begin();
30 }
```

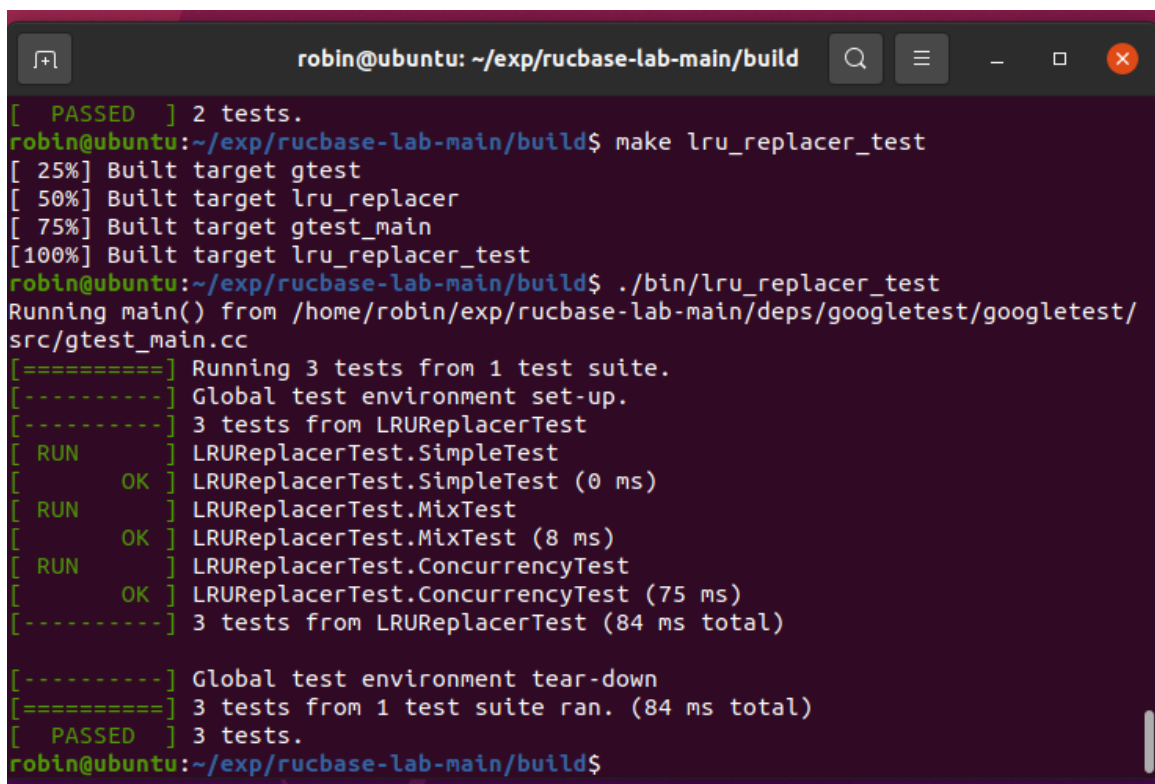
Listing 5: LRUReplacer 核心操作

- **并发控制:** 为保证多线程环境下的数据结构一致性, 所有公共方法 (`victim`, `pin`, `unpin`, `Size`) 的实现都在函数开头使用了 `std::scoped_lock` 对内部互斥量 `latch_` 进行加锁, 确保了操作的原子性。

测试结果 在 `DiskManager` 测试通过的基础上, 对 `LRUReplacer` 进行了单元测试。执行了 `cmake` 命令 (代码6), 所有测试用例均顺利通过。

```
1 make lru_replacer_test
2 ./bin/lru_replacer_test
```

Listing 6: LRUReplacer cmake



```

robin@ubuntu: ~/exp/rucbase-lab-main/build
[ PASSED ] 2 tests.
robin@ubuntu:~/exp/rucbase-lab-main/build$ make lru_replacer_test
[ 25%] Built target gtest
[ 50%] Built target lru_replacer
[ 75%] Built target gtest_main
[100%] Built target lru_replacer_test
robin@ubuntu:~/exp/rucbase-lab-main/build$ ./bin/lru_replacer_test
Running main() from /home/robin/exp/rucbase-lab-main/deps/googletest/googletest/
src/gtest_main.cc
[=====] Running 3 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 3 tests from LRURewriterTest
[ RUN      ] LRURewriterTest.SimpleTest
[       OK ] LRURewriterTest.SimpleTest (0 ms)
[ RUN      ] LRURewriterTest.MixTest
[       OK ] LRURewriterTest.MixTest (8 ms)
[ RUN      ] LRURewriterTest.ConcurrencyTest
[       OK ] LRURewriterTest.ConcurrencyTest (75 ms)
[-----] 3 tests from LRURewriterTest (84 ms total)

[-----] Global test environment tear-down
[=====] 3 tests from 1 test suite ran. (84 ms total)
[ PASSED ] 3 tests.
robin@ubuntu:~/exp/rucbase-lab-main/build$

```

图 6: LRURewriter 单元测试通过截图

2.1.3 任务 1.3: 缓冲池管理器 (BufferPoolManager)

实现描述 BufferPoolManager 是任务一的集大成者，它将 DiskManager 和 LRURewriter 组合起来，为上层提供统一的、高效的页面存取服务。

- **核心逻辑:** `fetch_page()` 是其最核心的函数之一。当请求一个页面时，首先查询 `page_table_` 哈希表。若页面已在缓冲池中（命中），则增加其 `pin_count_`，调用 `replacer->pin()` 防止其被淘汰，并直接返回页面指针。若未命中，则需从磁盘加载。此时调用私有辅助函数 `find_victim_page()`，该函数优先从 `free_list_` 获取空闲帧；若无空闲帧，则调用 `replacer->victim()` 淘汰一个页面。在获取到可用帧后，若该帧原先存有脏页 (`is_dirty_` 为 `true`)，则先调用 `disk_manager->write_page()` 将其写回磁盘。最后，从磁盘读入请求的页面内容，更新 `page_table_` 和 `Page` 对象的元数据，并返回页面。

```

1 Page* BufferPoolManager::fetch_page(PageId page_id) {
2     std::scoped_lock lock{latch_};
3     // 1. Search in page_table_
4     if (page_table_.count(page_id)) {
5         frame_id_t frame_id = page_table_[page_id];
6         Page* page = &pages_[frame_id];
7         page->pin_count_++;
8         replacer->pin(frame_id);
9         return page;
10    }
11    // 2. Not in buffer, find a victim page

```

```
12     frame_id_t frame_id;
13     if (!find_victim_page(&frame_id)) {
14         return nullptr;
15     }
16     Page* victim_page = &pages_[frame_id];
17     // 3. If victim is dirty, write back to disk
18     if (victim_page->is_dirty_) {
19         disk_manager_->write_page(victim_page->get_page_id().fd,
20                                   victim_page->get_page_id().page_no,
21                                   victim_page->get_data(), PAGE_SIZE);
22     }
23     // 4. Remove old page's mapping
24     if (victim_page->get_page_id().page_no != INVALID_PAGE_ID) {
25         page_table_.erase(victim_page->get_page_id());
26     }
27     // 5. Read new page from disk
28     disk_manager_->read_page(page_id.fd, page_id.page_no,
29                              victim_page->get_data(), PAGE_SIZE);
30     // 6. Update metadata
31     victim_page->id_ = page_id; // Friend access
32     victim_page->pin_count_ = 1;
33     victim_page->is_dirty_ = false;
34     page_table_[page_id] = frame_id;
35     replacer_->pin(frame_id);
36     return victim_page;
37 }
```

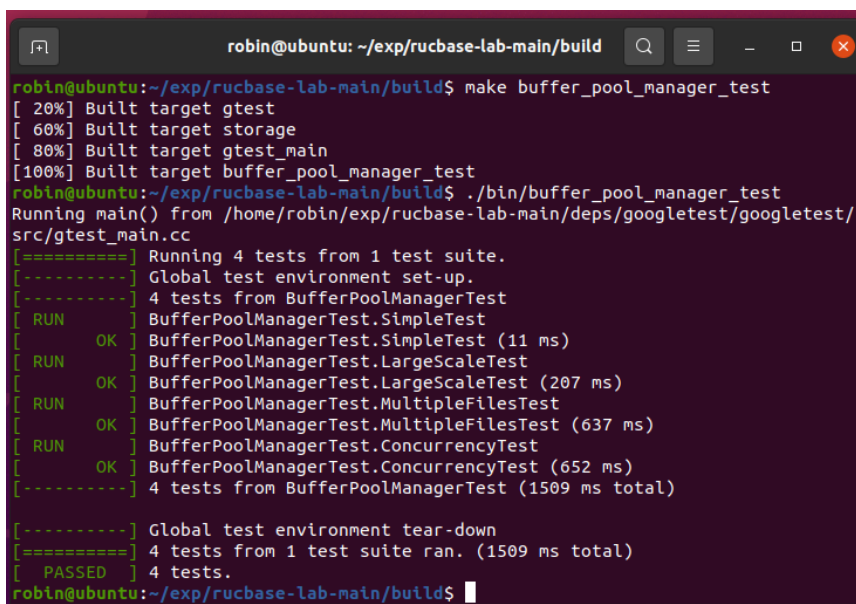
Listing 7: BufferPoolManager 的 fetch_page 核心逻辑

- **其他操作:** new_page() 的逻辑与 fetch_page() 的未命中路径类似, 但它调用的是 disk_manager_->allocate_page() 来创建新页。unpin_page() 负责将页面的 pin_count_ 减一, 并在其降为 0 时调用 replacer_->unpin()。
- **并发控制:** 所有公共接口都使用了 std::scoped_lock 进行保护, 保证了在多线程并发访问缓冲池时的数据结构一致性。

测试结果 在确保底层模块正确后, 对 BufferPoolManager 进行了测试。执行了 cmake 命令 (代码8), 所有测试用例均成功通过, 标志着任务一的核心功能已全部正确实现。

```
1 make buffer_pool_manager_test
2 ./bin/buffer_pool_manager_test
```

Listing 8: DiskManager cmake



```
robin@ubuntu: ~/exp/rucbase-lab-main/build
robin@ubuntu:~/exp/rucbase-lab-main/build$ make buffer_pool_manager_test
[ 20%] Built target gtest
[ 60%] Built target storage
[ 80%] Built target gtest_main
[100%] Built target buffer_pool_manager_test
robin@ubuntu:~/exp/rucbase-lab-main/build$ ./bin/buffer_pool_manager_test
Running main() from /home/robin/exp/rucbase-lab-main/deps/googletest/googletest/src/gtest_main.cc
[=====] Running 4 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 4 tests from BufferPoolManagerTest
[ RUN      ] BufferPoolManagerTest.SimpleTest
[ OK       ] BufferPoolManagerTest.SimpleTest (11 ms)
[ RUN      ] BufferPoolManagerTest.LargeScaleTest
[ OK       ] BufferPoolManagerTest.LargeScaleTest (207 ms)
[ RUN      ] BufferPoolManagerTest.MultipleFilesTest
[ OK       ] BufferPoolManagerTest.MultipleFilesTest (637 ms)
[ RUN      ] BufferPoolManagerTest.ConcurrencyTest
[ OK       ] BufferPoolManagerTest.ConcurrencyTest (652 ms)
[-----] 4 tests from BufferPoolManagerTest (1509 ms total)

[-----] Global test environment tear-down
[=====] 4 tests from 1 test suite ran. (1509 ms total)
[ PASSED  ] 4 tests.
robin@ubuntu:~/exp/rucbase-lab-main/build$
```

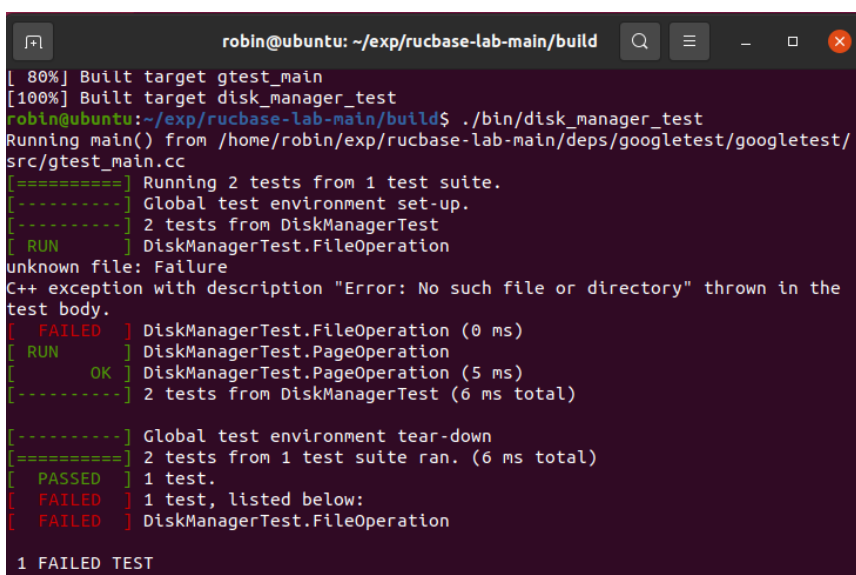
图 7: BufferPoolManager 单元测试通过截图

2.2 任务一遇到的问题及解决方案

在实现任务一的过程中，我遇到了两个主要的、具有代表性的技术挑战。这些问题的解决过程，极大地加深了我对系统级编程中接口契约、成员访问控制以及 API 适配重要性的理解。

2.2.1 DiskManager 的接口契约问题

问题描述与现象 在完成 DiskManager 的初步实现后，运行单元测试 `disk_manager_test` 时，出现了部分测试通过、部分失败的情况。`PageOperation` 测试可以顺利通过，但 `FileOperation` 测试点持续失败，终端抛出 `FileNotFoundException` 或底层的 `UnixError` 异常。



```
robin@ubuntu: ~/exp/rucbase-lab-main/build
[ 80%] Built target gtest_main
[100%] Built target disk_manager_test
robin@ubuntu:~/exp/rucbase-lab-main/build$ ./bin/disk_manager_test
Running main() from /home/robin/exp/rucbase-lab-main/deps/googletest/googletest/src/gtest_main.cc
[=====] Running 2 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 2 tests from DiskManagerTest
[ RUN      ] DiskManagerTest.FileOperation
unknown file: Failure
C++ exception with description "Error: No such file or directory" thrown in the
test body.
[ FAILED   ] DiskManagerTest.FileOperation (0 ms)
[ RUN      ] DiskManagerTest.PageOperation
[ OK       ] DiskManagerTest.PageOperation (5 ms)
[-----] 2 tests from DiskManagerTest (6 ms total)

[-----] Global test environment tear-down
[=====] 2 tests from 1 test suite ran. (6 ms total)
[ PASSED  ] 1 test.
[ FAILED  ] 1 test, listed below:
[ FAILED  ] DiskManagerTest.FileOperation

1 FAILED TEST
```

图 8: `disk_manager_test` 中 `FileOperation` 测试失败

问题分析 问题的关键在于理解测试代码 `disk_manager_test.cpp` 的真实意图。通过深入阅读其源码，我发现它对 `open_file` 和 `create_file` 函数的行为有着严格的“接口契约”（Interface Contract）要求。测试逻辑首先会尝试 `open_file` 一个不存在的文件，并期望程序能正确地抛出 `FileNotFoundError` 异常。

我最初的错误实现如下，它混淆了“打开”和“创建”的职责：

```
1 // 错误版本：尝试打开一个不存在的文件时，会直接创建它，而不是抛出异常
2 int DiskManager::open_file(const std::string &path) {
3     // ...
4     // O_CREAT 标志导致了与测试意图不符的行为
5     int fd = open(path.c_str(), O_RDWR | O_CREAT, 0666);
6     if (fd < 0) {
7         throw UnixError();
8     }
9     // ...
10 }
```

Listing 9: 错误的 `open_file` 实现

这个实现违反了测试用例的预期，即它没有在文件不存在时抛出 `FileNotFoundError`，而是创建了文件。这揭示了在系统编程中，精确地遵循 API 文档和测试用例所隐含的契约是至关重要的。

解决方案 解决方案是重构 `open_file` 函数，使其行为与接口契约完全一致。在函数开头，先调用 `is_file()` 来检查文件是否存在。只有当文件确实存在时，才调用 `open()` 系统调用；如果文件不存在，则立即抛出指定的异常。

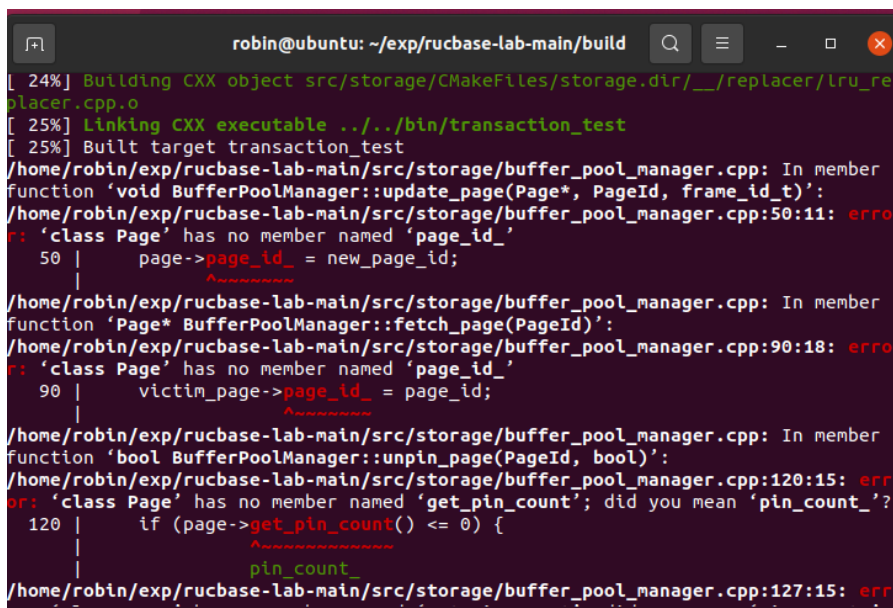
```
1 // 正确版本：严格遵守“只打开，不创建”的职责
2 int DiskManager::open_file(const std::string &path) {
3     if (path2fd_.count(path)) {
4         throw FileNotClosedError(path);
5     }
6     // 关键修正：在打开前先检查文件是否存在
7     if (!is_file(path)) {
8         throw FileNotFoundError(path);
9     }
10    // 仅使用 O_RDWR 标志，要求文件必须存在
11    int fd = open(path.c_str(), O_RDWR);
12    if (fd < 0) {
13        throw UnixError();
14    }
15    path2fd_[path] = fd;
16    fd2path_[fd] = path;
17    return fd;
18 }
```

Listing 10: 修正后的 `open_file` 实现

这个修改使得 `DiskManager` 的行为完全符合其接口定义和测试预期，从而顺利通过了所有单元测试。

2.2.2 BufferPoolManager 与 Page 类的接口适配问题

问题描述与现象 在实现 BufferPoolManager 时，编译阶段出现了大量的编译错误。错误信息主要表现为无法访问 Page 类的 id_、pin_count_ 等成员，提示这些成员是私有的。



```
robin@ubuntu: ~/exp/rucbase-lab-main/build
[ 24%] Building CXX object src/storage/CMakeFiles/storage.dir/_/replacer/lru_replac
[ 25%] Linking CXX executable ../../bin/transaction_test
[ 25%] Built target transaction_test
/home/robin/exp/rucbase-lab-main/src/storage/buffer_pool_manager.cpp: In member
function 'void BufferPoolManager::update_page(Page*, PageId, frame_id_t)':
/home/robin/exp/rucbase-lab-main/src/storage/buffer_pool_manager.cpp:50:11: erro
r: 'class Page' has no member named 'page_id_'
   50 |     page->page_id_ = new_page_id;
       |           ^~~~~~
/home/robin/exp/rucbase-lab-main/src/storage/buffer_pool_manager.cpp: In member
function 'Page* BufferPoolManager::fetch_page(PageId)':
/home/robin/exp/rucbase-lab-main/src/storage/buffer_pool_manager.cpp:90:18: erro
r: 'class Page' has no member named 'page_id_'
   90 |     victim_page->page_id_ = page_id;
       |                   ^~~~~~
/home/robin/exp/rucbase-lab-main/src/storage/buffer_pool_manager.cpp: In member
function 'bool BufferPoolManager::unpin_page(PageId, bool)':
/home/robin/exp/rucbase-lab-main/src/storage/buffer_pool_manager.cpp:120:15: err
or: 'class Page' has no member named 'get_pin_count'; did you mean 'pin_count_'?
  120 |     if (page->get_pin_count() <= 0) {
       |               ^~~~~~
/home/robin/exp/rucbase-lab-main/src/storage/buffer_pool_manager.cpp:127:15: err
```

图 9: BufferPoolManager 编译时出现成员访问错误

问题分析 这个问题的根源在于对 C++ 类成员访问控制和友元 (friend) 机制的理解不够深入。起初，我尝试通过公有接口访问这些成员，但发现框架并未提供对应的 setter 方法。我的错误代码片段如下：

```
1 // 错误尝试 1: 假设有 getter/setter
2 page->set_id(new_page_id); // 编译错误: Page 类没有 set_id 方法
3
4 // 错误尝试 2: 误用成员变量名
5 page->page_id_ = new_page_id; // 编译错误: Page 类没有名为 page_id_ 的成员
```

Listing 11: 错误的 Page 成员访问

通过仔细检查头文件 page.h，最终发现了两个关键点：

1. Page 类中明确声明了 friend class BufferPoolManager;，这赋予了 BufferPoolManager 直接访问其所有私有成员的特权。
2. 引起编译错误的另一个原因是，我错误地使用了 page->page_id_ 来访问页面 ID，而 page.h 中定义的实际成员变量名是 page->id_。

解决方案 在明确了友元关系和正确的成员变量名后，问题迎刃而解。buffer_pool_manager.cpp 修正了所有对 Page 对象成员的访问代码，改为直接通过指针访问其私有成员，因为友元关系允许这样做。

```
1 // 正确的实现：利用 friend 关系直接访问私有成员
2 victim_page->id_ = page_id; // 正确的成员名是 id_
3 victim_page->pin_count_ = 1;
4 victim_page->is_dirty_ = false;
```

Listing 12: 修正后的 Page 成员访问

这次调试经历让我深刻认识到，在进行系统级开发时，必须仔细阅读并理解框架提供的所有头文件，特别是类与类之间的协作关系（如友元），才能正确地使用其接口和内部成员。

3 任务二：记录管理器实现

在完成了底层的缓冲池管理后，任务二的目标是在页面之上，构建一个能够理解和操作逻辑记录（Record）的记录管理器。这要求我们在定长的页面内设计并维护一套元数据结构，用于跟踪记录的存储位置和页面的使用状态。

3.1 记录管理器的形象化理解

为了更直观地理解记录管理器的工作原理，我们可以将其比作一个管理大型图书馆的系统。在这个比喻中，各个组件的对应关系如下：

- **数据文件 (table.dbf)**: 整座图书馆。
- **一条记录 (Record)**: 一本书。
- **一个页面 (Page)**: 一个书架。
- **记录管理器 (RMFileHandle)**: 图书馆管理员。

管理员的目标是高效地找到书、存放新书、以及处理旧书。整个管理流程依赖于一套精心设计的“台账系统”。

总台账：文件头 (RmFileHdr) 在图书馆的入口处，管理员放置了一本总登记簿，这就是存储在文件第 0 页的**文件头**。它记录了图书馆最重要的全局信息：

- **“每本书有多厚” (record_size)**: 明确每本书（记录）的尺寸，以便计算一个书架能容纳多少本书。
- **“图书馆共有多少书架” (num_pages)**: 记录图书馆的总规模。
- **“下一个有空位的书架在哪？” (first_free_page_no)**: 这是最高效的设计之一。管理员无需从头寻找空位，而是直接查看总台账，上面清晰地指明了第一个有空闲位置的书架编号。这构成了一个“空闲书架链表”的入口。

书架卡片：页面头 (RmPageHdr) 与位图 (Bitmap) 当管理员走到一个具体的书架前，他需要查阅该书架自身的详细信息。每个书架（页面）都附带一张记录其状态的“小卡片”，这张卡片由**页面头**和**位图**组成。

- **页面头**: 记录了这个书架当前的状态，如“已存放书籍数量” (num_records)，以及“如果我满了，下一个空闲书架是几号？” (next_free_page_no)，用于维护空闲书架链。

- **位图**: 这好比书架上每个预留位置都贴了一个标签。标签为“绿色”(比特位为 0) 表示“此位可放新书”; 标签为“红色”(比特位为 1) 表示“此位已有书”。这使得管理员可以快速确定书架上的具体空闲位置。

3.1.1 管理员的核心工作流程

插入一本书 (insert_record) 一位读者捐赠一本新书, 管理员的操作流程如下:

1. **查阅总台账**: 管理员首先查看总台账上的 `first_free_page_no`。假设它指向 8 号书架。
2. **定位书架**: 管理员直接前往 8 号书架, 并获取其“书架卡片”(即获取页面的 `RmPageHandle`)。
3. **寻找空位**: 通过扫描书架上的“标签”(位图), 管理员找到了第一个“绿色”标签, 例如在第 3 个位置。
4. **放置新书**: 管理员将新书放入第 3 个位置, 并立刻将该位置的标签翻为“红色”(调用 `Bitmap::set()`)。同时, 更新卡片上的书籍数量。
5. **处理书架变满**: 如果此次放置后, 8 号书架的所有位置都变为了“红色”, 管理员会查看 8 号书架卡片上记录的“下一个空闲书架”(比如是 15 号), 然后回到总台, 将总台账上的 `first_free_page_no` 从 8 号更新为 15 号。这样, 空闲链表的头指针就指向了下一个真正有空位的书架。

处理一本旧书 (delete_record) 图书馆需要淘汰 5 号书架的第 2 本书。

1. **定位并移除**: 管理员径直走到 5 号书架, 将第 2 本书取走, 并将其位置的标签从“红色”翻回“绿色”(调用 `Bitmap::reset()`)。
2. **处理书架变空闲**: 一个关键的场景是: 如果 5 号书架在此之前是满的, 但现在因为移除了这本书而出现了空位, 它就必须被重新加入到“空闲书架链”中。管理员会执行一个“头插法”操作:
 - 在 5 号书架的卡片上, 将其“下一个空闲书架”指针指向当前总台账记录的 `first_free_page_no` (例如 15 号)。
 - 然后, 回到总台, 将总台账的 `first_free_page_no` 更新为 5 号。

如此一来, 空闲链表就从 `15 -> ...` 变成了 `5 -> 15 -> ...`, 新来的书籍会被优先放置到这个刚有空位的 5 号书架。

图书盘点 (RMScan) 进行全馆盘点时, 管理员(即迭代器 `RMScan`)从 1 号书架开始, 依次检查每个书架。对于每个书架, 他只关心所有贴着“红色”标签的位置, 并将这些位置上的书籍信息记录下来。完成一个书架后, 便移步至下一个, 直到所有书架都被检查完毕。

记录管理器本质上是一个高效的簿记系统。它通过维护全局的“总台账”和局部的“书架卡片”，实现了对大量记录的高效、有序的管理。

3.2 实验过程与实现细节

3.2.1 任务 2.1: 记录操作 (RMFileHandle)

实现描述 RMFileHandle 的实现核心在于对文件头 `file_hdr_` 和每个页面的页面头 `page_hdr` 的精确维护。

- **插入 (insert_record):** 通过私有辅助函数 `create_page_handle` 获取空闲页面（优先使用 `file_hdr_.first_free_page_no` 指向的空闲链表），然后利用框架提供的 `Bitmap::first_bit` 函数高效地找到空闲槽位。
- **删除 (delete_record):** 通过 `Bitmap::reset` 清除记录位，并检查页面是否从“满”变为“非满”，若是则调用私有辅助函数 `release_page_handle` 将其加入空闲链表。
- **持久化:** 为确保在 SimpleTest 的随机文件重开场景下状态不丢失,在所有修改 `file_hdr_` 的关键路径（如页面变满、创建新页）后，都内联了将 `file_hdr_` 写回到缓冲池第 0 页并标记为脏页的逻辑，这是保证测试通过的关键。

```
1 Rid RmFileHandle::insert_record(char* buf, Context* context) {
2     // 1. 获取一个有空闲空间的页面句柄，优先从空闲链表获取，若无则创建新页
3     RmPageHandle page_handle = create_page_handle();
4
5     // 2. 在页面位图中找到第一个空闲的槽位 (bit为0的位置)
6     int slot_no = Bitmap::first_bit(false, page_handle.bitmap,
7                                     file_hdr_.num_records_per_page);
8     assert(slot_no < file_hdr_.num_records_per_page); // 确保找到了有效的槽位
9
10    // 3. 将记录数据拷贝到该槽位的物理地址
11    memcpy(page_handle.get_slot(slot_no), buf, file_hdr_.record_size);
12
13    // 4. 更新页面元数据：将槽位标记为已使用，并增加页面中的记录数
14    Bitmap::set(page_handle.bitmap, slot_no);
15    page_handle.page_hdr->num_records++;
16
17    // 5. 检查页面是否因此次插入而变满
18    if (page_handle.page_hdr->num_records == file_hdr_.num_records_per_page) {
19        // 页面已满，将其从空闲链表中移除
20        file_hdr_.first_free_page_no = page_handle.page_hdr->next_free_page_no;
21
22        // 关键：立即将文件头的修改持久化到缓冲池的第0页
23        // 这是为了防止因文件重开导致的状态丢失问题
24        Page* hdr_page = buffer_pool_manager->fetch_page({fd_, RM_FILE_HDR_PAGE});
25        assert(hdr_page != nullptr);
26        memcpy(hdr_page->get_data(), &file_hdr_, sizeof(file_hdr_));
27        buffer_pool_manager->unpin_page({fd_, RM_FILE_HDR_PAGE}, true);
28    }
```

```

29
30 // 6. 构造新记录的 Rid (页面号, 槽位号)
31 Rid rid{page_handle.page->get_page_id().page_no, slot_no};
32
33 // 7. 完成操作, unpin数据页, 并标记为脏页
34 buffer_pool_manager->unpin_page(page_handle.page->get_page_id(), true);
35 return rid;
36 }

```

Listing 13: RMFileHandle 插入记录与文件头持久化

测试结果 经过对状态持久化问题的深入调试, 最终 `record_manager_test` 的所有测试用例, 包括严苛的 `SimpleTest` 和 `MultipleFilesTest`, 均成功通过。

```

robin@ubuntu: ~/exp/rucbase-lab-main/build
[ 62%] Built target transaction
[ 70%] Built target system
[ 83%] Built target record
[ 91%] Built target gtest_main
[100%] Built target record_manager_test
robin@ubuntu:~/exp/rucbase-lab-main/build$ ./bin/record_manager_test
Running main() from /home/robin/exp/rucbase-lab-main/deps/googletest/googletest/src/gtest_main.cc
[====] Running 2 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 2 tests from RecordManagerTest
[ RUN ] RecordManagerTest.SimpleTest
insert 431
delete 255
update 314
[ OK ] RecordManagerTest.SimpleTest (295 ms)
[ RUN ] RecordManagerTest.MultipleFilesTest
[ OK ] RecordManagerTest.MultipleFilesTest (98 ms)
[-----] 2 tests from RecordManagerTest (394 ms total)

[-----] Global test environment tear-down
[====] 2 tests from 1 test suite ran. (394 ms total)
[ PASSED ] 2 tests.
robin@ubuntu:~/exp/rucbase-lab-main/build$

```

图 10: RecordManager 单元测试通过截图

3.2.2 任务 2.2: 记录迭代器 (RMScan)

实现描述 RMScan 实现了对文件中所有记录的顺序扫描。其 `next()` 方法通过一个外层循环遍历所有数据页 (从第 1 页开始), 内层逻辑则利用框架提供的 `Bitmap::next_bit(true, ...)` 函数来高效地在当前页的位图中查找下一个有效记录的槽位。当内层查找不到时 (函数返回的槽位号超出范围), 则移动到下一页, 并重置槽位搜索的起始点。当所有页面都扫描完毕后, 将内部 `rid_` 设置为结束标志 `RM_NO_PAGE`。实现中特别注意了对每个获取到的 `page_handle` 进行及时的 `unpin` 操作, 防止了页面资源泄漏。

```

1 void RmScan::next() {
2     int current_page = rid_.page_no;
3     int current_slot = rid_.slot_no;
4     while(current_page < file_handle->file_hdr_.num_pages) {

```

```

5      // 在当前页内查找下一个有效的记录
6      current_slot = Bitmap::next_bit(true, file_handle->fetch_page_handle(
          current_page).bitmap,
7                                     file_handle->file_hdr_.num_records_per_page,
                                     current_slot);
8      if (current_slot < file_handle->file_hdr_.num_records_per_page) { // 找到了
9          rid_ = {current_page, current_slot};
10         return;
11     }
12     current_page++; // 当前页扫描完毕, 移动到下一页
13     current_slot = -1; // 从下一页的第一个槽位开始
14 }
15 // 如果循环结束仍未找到, 说明已到达文件末尾
16 rid_ = {RM_NO_PAGE, -1};
17 }

```

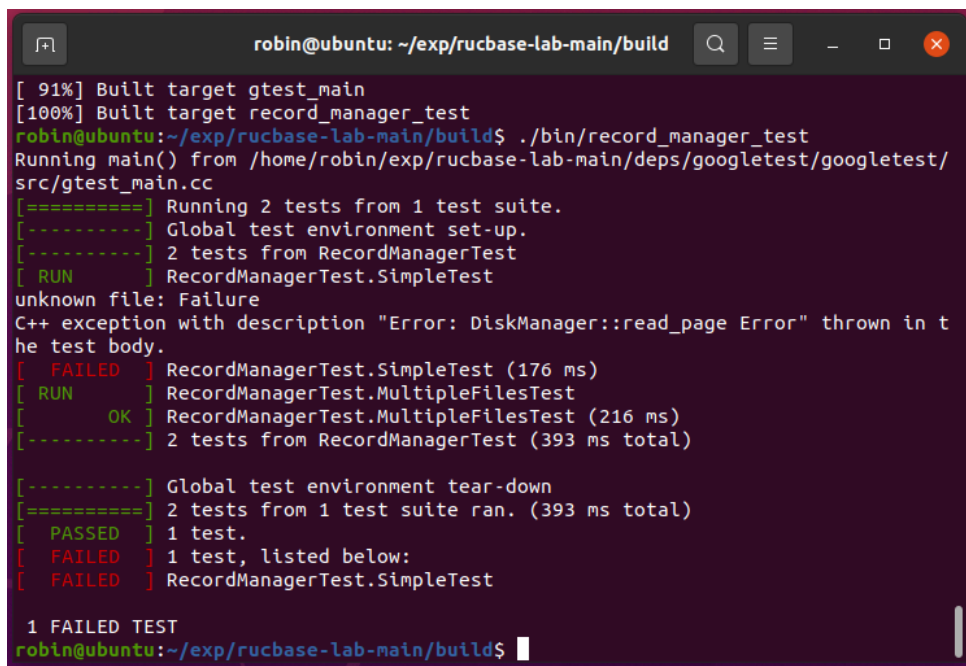
Listing 14: RMScan 找到文件中下一个存放记录的位置

测试结果 (record_manager_test 的通过已经验证了 RMScan 的正确性。)

3.3 任务二遇到的问题及解决方案

任务二的核心挑战集中在 SimpleTest 测试用例上, 它暴露了系统在复杂并发和状态变动下的一个深层次逻辑漏洞。

3.3.1 核心问题: SimpleTest 中因状态不一致导致的越界读取



```

robin@ubuntu: ~/exp/rucbase-lab-main/build
[ 91%] Built target gtest_main
[100%] Built target record_manager_test
robin@ubuntu:~/exp/rucbase-lab-main/build$ ./bin/record_manager_test
Running main() from /home/robin/exp/rucbase-lab-main/deps/googletest/googletest/src/gtest_main.cc
[=====] Running 2 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 2 tests from RecordManagerTest
[ RUN    ] RecordManagerTest.SimpleTest
unknown file: Failure
C++ exception with description "Error: DiskManager::read_page Error" thrown in the test body.
[ FAILED ] RecordManagerTest.SimpleTest (176 ms)
[ RUN    ] RecordManagerTest.MultipleFilesTest
[ OK     ] RecordManagerTest.MultipleFilesTest (216 ms)
[-----] 2 tests from RecordManagerTest (393 ms total)

[-----] Global test environment tear-down
[=====] 2 tests from 1 test suite ran. (393 ms total)
[ PASSED ] 1 test.
[ FAILED ] 1 test, listed below:
[ FAILED ] RecordManagerTest.SimpleTest

1 FAILED TEST
robin@ubuntu:~/exp/rucbase-lab-main/build$

```

图 11: SimpleTest 因 read_page 错误而失败

问题描述与现象 在 RecordManager 的开发过程中，遇到了一个最顽固的问题：‘MultipleFilesTest’ 能够通过，但 ‘SimpleTest’ 反复失败，并抛出底层的 DiskManager::read_page Error 异常。这表明上层逻辑请求读取一个物理上不存在的页面。

问题分析 通过在代码中添加调试日志，最终定位到问题的根源。SimpleTest 的测试逻辑会以 50 次为一轮，频繁地关闭并重新打开同一个文件句柄。

```
1 // Randomly re-open file
2 if (round % 50 == 0) {
3     rm_manager->close_file(file_handle.get());
4     file_handle = rm_manager->open_file(filename);
5 }
6 check_equal(file_handle.get(), mock);
```

Listing 15: SimpleTest 中的关键测试逻辑

问题在于，当 rm_manager->close_file() 被调用时，对应的 RmFileHandle 对象被销毁。我在该对象内存中维护的 file_hdr_ 副本，如果其上的修改（如因创建新页而增加的 num_pages）没有被写回到磁盘（或缓冲池中的第 0 页），那么这些修改就会丢失。随后，当 rm_manager->open_file() 创建一个新的 RmFileHandle 对象时，它会从磁盘的第 0 页重新读取文件头，此时读到的是一个**过时的、未被更新的**元数据。

最终，当 check_equal 函数基于这个过时的、较小的 num_pages 值去访问一个实际上已存在但逻辑上“不可见”的页面时，就会向底层传递一个越界的页面号，从而引发了 DiskManager::read_page Error。

解决方案 最终的解决方案是采取了最健壮但有一定冗余的**内联持久化**策略。在每个会修改 file_hdr_ 的成员函数（如 insert_record 中页面变满时，delete_record 中页面变非满时，以及 create_new_page_handle）的内部，都直接嵌入了将 file_hdr_ 写回到缓冲池第 0 页并标记为脏页的逻辑。

```
1 RmPageHandle RmFileHandle::create_new_page_handle() {
2     // ... (代码创建新页面, new_page_handle)
3
4     file_hdr_.num_pages++;
5     file_hdr_.first_free_page_no = new_page_id.page_no;
6
7     // Inlined persist logic:
8     Page* hdr_page = buffer_pool_manager->fetch_page({fd_, RM_FILE_HDR_PAGE});
9     assert(hdr_page != nullptr);
10    memcpy(hdr_page->get_data(), &file_hdr_, sizeof(file_hdr_));
11    buffer_pool_manager->unpin_page({fd_, RM_FILE_HDR_PAGE}, true);
12
13    return new_page_handle;
14 }
```

Listing 16: create_new_page_handle 中的内联持久化逻辑

这种方式虽然牺牲了一定的代码简洁性，但它确保了文件头状态的实时一致性，从根本上解决了因文件句柄重开导致的状态丢失问题，最终使得 SimpleTest 得以顺利通过。

4 实验总结与心得体会

本次 Rucbase 存储管理器的设计与实现，是一次极具挑战性与收获的实践经历。通过从最底层的磁盘文件抽象到上层的记录操作，我完整地构建了一个小型数据库存储引擎的基石。这不仅让我将《数据库系统原理》课程中学到的理论知识付诸实践，更让我对系统级编程的复杂性、严谨性和设计哲学有了前所未有的深刻理解。

对系统分层设计的理解 实验伊始，我按照 DiskManager、BufferPoolManager、RecordManager 的顺序自底向上进行开发。这个过程让我真切地体会到分层架构的优势。每一层都向其上层提供清晰、稳定的接口，并隐藏其内部复杂的实现细节。例如，RecordManager 无需关心页面是如何从磁盘加载到内存的，也无需关心 LRU 替换策略的具体运作，它只需要向 BufferPoolManager 请求或提交页面即可。这种关注点分离的设计大大降低了系统的复杂度，使得我们可以专注于当前层次的逻辑，也使得系统更易于维护和扩展。

状态一致性的重要性 在任务二中遇到的关于 file_hdr_ 的持久化问题，是本次实验给我上的最深刻的一课。最初，我仅仅在内存中更新文件头的状态（如 num_pages），而忽略了在关键时机将其写回。这导致在 SimpleTest 频繁重开文件的压力下，内存与磁盘（缓冲池）的状态出现了不一致，最终引发了连锁的、难以追踪的运行时错误。这个调试过程让我深刻地认识到，在任何有持久化需求的系统中，**状态的一致性与同步是保证系统正确性的生命线**。任何对核心元数据的内存修改，都必须伴随着一个明确、可靠的持久化策略，否则在对象生命周期结束或状态重载时，就会导致灾难性的后果。最终采用的“内联持久化”方案虽然在代码上有所冗余，但它用最直接的方式保证了状态的强一致性，是面对复杂测试场景时最稳妥的选择。

调试与问题定位能力的提升 本次实验也极大地锻炼了我的工程调试能力。面对复杂的底层系统，单纯的“看代码”往往难以发现问题。我学会了如何有效地利用工具和方法来定位错误：

- **单元测试驱动**: 严格遵循先通过底层模块测试，再测试上层模块的原则。这帮助我将问题范围从整个系统缩小到单个模块，甚至单个函数。
- **分析测试用例**: 当遇到测试失败时，我不再是盲目修改代码，而是先去仔细阅读测试代码（如 disk_manager_test.cpp 和 record_manager_test.cpp），理解其背后的测试逻辑和对被测接口的“契约”要求。这被证明是解决接口适配问题的最有效方法。
- **日志与调试信息**: 在解决‘SimpleTest’的顽固问题时，通过在关键路径上添加 std::cout 调试信息，我最终观察到了‘num_pages’状态回滚的现象，从而定位到了持久化问

题的根源。这让我认识到，在没有高级调试器的情况下，巧妙地使用日志是追踪复杂系统行为的有力武器。

对 C++ 语言特性的深入应用 除了数据库知识，本次实验也促使我更深入地应用和理解了 C++ 的许多高级特性。例如，为了实现高效的 LRU，我组合使用了 `std::unordered_map` 和 `std::list`；为了保证线程安全，我学习并使用了 `std::mutex` 和 `std::scoped_lock`；在面对编译错误时，我深入研究了 `friend` 关键字的作用，以及 `const_cast` 在特定场景下的合理用法。这些实践都远比课堂上学习理论知识要来得具体和深刻。

总而言之，Rucbase 实验一是一次从理论到实践的完美跨越。它不仅巩固了我的数据库核心知识，更全面地提升了我的系统设计、编程实现和工程调试能力。这次从反复失败到最终成功的经历，让我对系统编程的严谨性和复杂性有了全新的敬畏和认识，也为我后续学习更高级的数据库技术打下了坚实的基础。

A 附录：实验代码

A.1 disk_manager.cpp

```
1  /* Copyright (c) 2023 Renmin University of China
2  RMDB is licensed under Mulan PSL v2.
3  You can use this software according to the terms and conditions of the Mulan PSL v2.
4  You may obtain a copy of Mulan PSL v2 at:
5      http://license.coscl.org.cn/MulanPSL2
6  THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, WITHOUT WARRANTIES OF ANY KIND,
7  EITHER EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO NON-INFRINGEMENT,
8  MERCHANTABILITY OR FIT FOR A PARTICULAR PURPOSE.
9  See the Mulan PSL v2 for more details. */
10
11 #include "storage/disk_manager.h"
12
13 #include <assert.h>    // for assert
14 #include <string.h>    // for memset
15 #include <sys/stat.h>  // for stat
16 #include <unistd.h>    // for lseek
17 #include <fcntl.h>     // for O_CREAT
18
19 #include "defs.h"
20
21 DiskManager::DiskManager() { memset(fd2pageno_, 0, MAX_FD * (sizeof(std::atomic<page_id_t>) / sizeof(char))); }
22
23 /**
24  * @description: 将数据写入文件的指定磁盘页面中
25  * @param {int} fd 磁盘文件的文件句柄
26  * @param {page_id_t} page_no 写入目标页面的page_id
27  * @param {char} *offset 要写入磁盘的数据
28  * @param {int} num_bytes 要写入磁盘的数据大小
29  */
30 void DiskManager::write_page(int fd, page_id_t page_no, const char *offset, int num_bytes) {
31     // Todo:
32     // 1.lseek()定位到文件头，通过(fd,page_no)可以定位指定页面及其在磁盘文件中的偏移量
33     // 2.调用write()函数
34     // 注意write返回值与num_bytes不等时 throw InternalError("DiskManager::write_page
35         Error");
36     off_t disk_offset = static_cast<off_t>(page_no) * PAGE_SIZE;
37     if (lseek(fd, disk_offset, SEEK_SET) != disk_offset) {
38         throw UnixError();
39     }
40     ssize_t bytes_written = write(fd, offset, num_bytes);
41     if (bytes_written != num_bytes) {
42         throw InternalError("DiskManager::write_page_Error");
43     }
44 }
45
46 /**
47  * @description: 读取文件中指定编号的页面中的部分数据到内存中
```

```

47  * @param {int} fd 磁盘文件的文件句柄
48  * @param {page_id_t} page_no 指定的页面编号
49  * @param {char} *offset 读取的内容写入到offset中
50  * @param {int} num_bytes 读取的数据量大小
51  */
52 void DiskManager::read_page(int fd, page_id_t page_no, char *offset, int num_bytes) {
53     // Todo:
54     // 1.lseek()定位到文件头, 通过(fd,page_no)可以定位指定页面及其在磁盘文件中的偏移量
55     // 2.调用read()函数
56     // 注意read返回值与num_bytes不等时, throw InternalError("DiskManager::read_page Error
57     // ");
58     off_t disk_offset = static_cast<off_t>(page_no) * PAGE_SIZE;
59     if (lseek(fd, disk_offset, SEEK_SET) != disk_offset) {
60         throw UnixError();
61     }
62     ssize_t bytes_read = read(fd, offset, num_bytes);
63     if (bytes_read != num_bytes) {
64         throw InternalError("DiskManager::read_page_Error");
65     }
66 }
67 /**
68  * @description: 分配一个新的页号
69  * @return {page_id_t} 分配的新页号
70  * @param {int} fd 指定文件的文件句柄
71  */
72 page_id_t DiskManager::allocate_page(int fd) {
73     // 简单的自增分配策略, 指定文件的页面编号加1
74     assert(fd >= 0 && fd < MAX_FD);
75     return fd2pageno_[fd]++;
76 }
77
78 void DiskManager::deallocate_page(__attribute__((unused)) page_id_t page_id) {}
79
80 bool DiskManager::is_dir(const std::string& path) {
81     struct stat st;
82     return stat(path.c_str(), &st) == 0 && S_ISDIR(st.st_mode);
83 }
84
85 void DiskManager::create_dir(const std::string &path) {
86     // Create a subdirectory
87     std::string cmd = "mkdir_" + path;
88     if (system(cmd.c_str()) < 0) { // 创建一个名为path的目录
89         throw UnixError();
90     }
91 }
92
93 void DiskManager::destroy_dir(const std::string &path) {
94     std::string cmd = "rm_r_" + path;
95     if (system(cmd.c_str()) < 0) {
96         throw UnixError();
97     }
98 }
99

```

```
100 /**
101  * @description: 判断指定路径文件是否存在
102  * @return {bool} 若指定路径文件存在则返回 true
103  * @param {string} &path 指定路径文件
104  */
105 bool DiskManager::is_file(const std::string &path) {
106     // 用 struct stat 获取文件信息
107     struct stat st;
108     return stat(path.c_str(), &st) == 0 && S_ISREG(st.st_mode);
109 }
110
111 /**
112  * @description: 用于创建指定路径文件
113  * @return {*}
114  * @param {string} &path
115  */
116 void DiskManager::create_file(const std::string &path) {
117     // Todo:
118     // 调用 open() 函数, 使用 O_CREAT 模式
119     // 注意不能重复创建相同文件
120     if (is_file(path)) {
121         throw FileExistsError(path);
122     }
123     int fd = open(path.c_str(), O_CREAT, 0666);
124     if (fd < 0) {
125         throw UnixError();
126     }
127     close(fd);
128 }
129
130 /**
131  * @description: 删除指定路径的文件
132  * @param {string} &path 文件所在路径
133  */
134 void DiskManager::destroy_file(const std::string &path) {
135     // Todo:
136     // 调用 unlink() 函数
137     // 注意不能删除未关闭的文件
138     if (path2fd_.count(path)) {
139         throw FileNotClosedError(path);
140     }
141     if (!is_file(path)) {
142         throw FileNotFoundError(path);
143     }
144     if (unlink(path.c_str()) != 0) {
145         throw UnixError();
146     }
147 }
148
149
150 /**
151  * @description: 打开指定路径文件
152  * @return {int} 返回打开的文件的文件句柄
153  * @param {string} &path 文件所在路径
```

```
154 */
155 int DiskManager::open_file(const std::string &path) {
156     // Todo:
157     // 调用 open() 函数, 使用 O_RDWR 模式
158     // 注意不能重复打开相同文件, 并且需要更新文件打开列表
159     if (path2fd_.count(path)) {
160         throw FileNotClosedError(path);
161     }
162     if (!is_file(path)) {
163         throw FileNotFoundError(path);
164     }
165     int fd = open(path.c_str(), O_RDWR);
166     if (fd < 0) {
167         throw UnixError();
168     }
169     path2fd_[path] = fd;
170     fd2path_[fd] = path;
171     return fd;
172 }
173
174 /**
175  * @description: 用于关闭指定路径文件
176  * @param {int} fd 打开的文件的文件句柄
177  */
178 void DiskManager::close_file(int fd) {
179     // Todo:
180     // 调用 close() 函数
181     // 注意不能关闭未打开的文件, 并且需要更新文件打开列表
182     if (!fd2path_.count(fd)) {
183         throw FileNotOpenError(fd);
184     }
185     std::string path = fd2path_[fd];
186     if (close(fd) != 0) {
187         throw UnixError();
188     }
189     path2fd_.erase(path);
190     fd2path_.erase(fd);
191 }
192
193
194 /**
195  * @description: 获得文件的大小
196  * @return {int} 文件的大小
197  * @param {string} &file_name 文件名
198  */
199 int DiskManager::get_file_size(const std::string &file_name) {
200     struct stat stat_buf;
201     int rc = stat(file_name.c_str(), &stat_buf);
202     return rc == 0 ? stat_buf.st_size : -1;
203 }
204
205 /**
206  * @description: 根据文件句柄获得文件名
207  * @return {string} 文件句柄对应文件的文件名
```

```
208  * @param {int} fd 文件句柄
209  */
210  std::string DiskManager::get_file_name(int fd) {
211      if (!fd2path_.count(fd)) {
212          throw FileNotOpenError(fd);
213      }
214      return fd2path_[fd];
215  }
216
217  /**
218   * @description: 获得文件名对应的文件句柄
219   * @return {int} 文件句柄
220   * @param {string} &file_name 文件名
221   */
222  int DiskManager::get_file_fd(const std::string &file_name) {
223      if (!path2fd_.count(file_name)) {
224          return open_file(file_name);
225      }
226      return path2fd_[file_name];
227  }
228
229
230  /**
231   * @description: 读取日志文件内容
232   * @return {int} 返回读取的数据量, 若为-1说明读取数据的起始位置超过了文件大小
233   * @param {char} *log_data 读取内容到log_data中
234   * @param {int} size 读取的数据量大小
235   * @param {int} offset 读取的内容在文件中的位置
236   */
237  int DiskManager::read_log(char *log_data, int size, int offset) {
238      // read log file from the previous end
239      if (log_fd_ == -1) {
240          log_fd_ = open_file(LOG_FILE_NAME);
241      }
242      int file_size = get_file_size(LOG_FILE_NAME);
243      if (offset > file_size) {
244          return -1;
245      }
246
247      size = std::min(size, file_size - offset);
248      if(size == 0) return 0;
249      lseek(log_fd_, offset, SEEK_SET);
250      ssize_t bytes_read = read(log_fd_, log_data, size);
251      assert(bytes_read == size);
252      return bytes_read;
253  }
254
255
256  /**
257   * @description: 写日志内容
258   * @param {char} *log_data 要写入的日志内容
259   * @param {int} size 要写入的内容大小
260   */
261  void DiskManager::write_log(char *log_data, int size) {
```



```
262     if (log_fd_ == -1) {
263         log_fd_ = open_file(LOG_FILE_NAME);
264     }
265
266     // write from the file_end
267     lseek(log_fd_, 0, SEEK_END);
268     ssize_t bytes_write = write(log_fd_, log_data, size);
269     if (bytes_write != size) {
270         throw UnixError();
271     }
272 }
```

Listing 17: disk_manager.cpp

A.2 lru_replacer.cpp

```
1  /* Copyright (c) 2023 Renmin University of China
2  RMDB is licensed under Mulan PSL v2.
3  You can use this software according to the terms and conditions of the Mulan PSL v2.
4  You may obtain a copy of Mulan PSL v2 at:
5      http://license.coscl.org.cn/MulanPSL2
6  THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, WITHOUT WARRANTIES OF ANY KIND,
7  EITHER EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO NON-INFRINGEMENT,
8  MERCHANTABILITY OR FIT FOR A PARTICULAR PURPOSE.
9  See the Mulan PSL v2 for more details. */
10
11 #include "lru_replacer.h"
12
13 LRUReplacer::LRUReplacer(size_t num_pages) { max_size_ = num_pages; }
14
15 LRUReplacer::~LRUReplacer() = default;
16
17 /**
18  * @description: 使用LRU策略删除一个victim frame, 并返回该frame的id
19  * @param {frame_id_t*} frame_id 被移除的frame的id, 如果没有frame被移除返回nullptr
20  * @return {bool} 如果成功淘汰了一个页面则返回true, 否则返回false
21  */
22 bool LRUReplacer::victim(frame_id_t* frame_id) {
23     // C++17 std::scoped_lock
24     // 它能够避免死锁发生, 其构造函数能够自动进行上锁操作, 析构函数会对互斥量进行解锁操作, 保证线程安全。
25     std::scoped_lock lock{latch_}; // 如果编译报错可以替换成其他lock
26
27     // Todo:
28     // 利用lru_replacer中的LRUlist_, LRUHash_实现LRU策略
29     // 选择合适的frame指定为淘汰页面, 赋值给*frame_id
30     if (LRUlist_.empty()) {
31         return false;
32     }
33     *frame_id = LRUlist_.front();
34     LRUhash_.erase(*frame_id);
35     LRUlist_.pop_front();
```

```

36     return true;
37 }
38
39 /**
40  * @description: 固定指定的 frame, 即该页面无法被淘汰
41  * @param {frame_id_t} 需要固定的 frame 的 id
42  */
43 void LRURemplacer::pin(frame_id_t frame_id) {
44     std::scoped_lock lock{latch_};
45     // Todo:
46     // 固定指定 id 的 frame
47     // 在数据结构中移除该 frame
48     auto it = LRUhash_.find(frame_id);
49     if (it != LRUhash_.end()) {
50         LRUlist_.erase(it->second);
51         LRUhash_.erase(it);
52     }
53 }
54
55 /**
56  * @description: 取消固定一个 frame, 代表该页面可以被淘汰
57  * @param {frame_id_t} frame_id 取消固定的 frame 的 id
58  */
59 void LRURemplacer::unpin(frame_id_t frame_id) {
60     // Todo:
61     // 支持并发锁
62     // 选择一个 frame 取消固定
63     std::scoped_lock lock{latch_};
64     if (LRUhash_.count(frame_id)) {
65         return;
66     }
67     LRUlist_.push_back(frame_id);
68     LRUhash_[frame_id] = --LRUlist_.end();
69 }
70
71 /**
72  * @description: 获取当前 replacer 中可以被淘汰的页面数量
73  */
74 size_t LRURemplacer::Size() { return LRUlist_.size(); }

```

Listing 18: lru_replacer.cpp

A.3 buffer_pool_manager.cpp

```

1  /* Copyright (c) 2023 Renmin University of China
2  RMDB is licensed under Mulan PSL v2.
3  You can use this software according to the terms and conditions of the Mulan PSL v2.
4  You may obtain a copy of Mulan PSL v2 at:
5      http://license.coscl.org.cn/MulanPSL2
6  THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, WITHOUT WARRANTIES OF ANY KIND,
7  EITHER EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO NON-INFRINGEMENT,
8  MERCHANTABILITY OR FIT FOR A PARTICULAR PURPOSE.

```

```

9  See the Mulan PSL v2 for more details. */
10
11 #include "buffer_pool_manager.h"
12
13 /**
14  * @description: 从free_list或replacer中得到可淘汰帧页的 *frame_id
15  * @return {bool} true: 可替换帧查找成功, false: 可替换帧查找失败
16  * @param {frame_id_t*} frame_id 帧页id指针, 返回成功找到的可替换帧id
17  */
18 bool BufferPoolManager::find_victim_page(frame_id_t* frame_id) {
19     // Todo:
20     // 1 使用BufferPoolManager::free_list_判断缓冲池是否已满需要淘汰页面
21     // 1.1 未满获得frame
22     // 1.2 已满使用lru_replacer中的方法选择淘汰页面
23     if (!free_list_.empty()) {
24         *frame_id = free_list_.front();
25         free_list_.pop_front();
26         return true;
27     }
28     return replacer_>victim(frame_id);
29 }
30
31 /**
32  * @description: 更新页面数据, 如果为脏页则需写入磁盘, 再更新为新页面, 更新page元数据(
33     data, is_dirty, page_id)和page table
34  * @param {Page*} page 写回页指针
35  * @param {PageId} new_page_id 新的page_id
36  * @param {frame_id_t} new_frame_id 新的帧frame_id
37  */
38 void BufferPoolManager::update_page(Page *page, PageId new_page_id, frame_id_t
39     new_frame_id) {
40     // Todo:
41     // 1 如果是脏页, 写回磁盘, 并且把dirty置为false
42     // 2 更新page table
43     // 3 重置page的data, 更新page id
44     if (page->is_dirty_) {
45         disk_manager->write_page(page->get_page_id().fd, page->get_page_id().page_no,
46             page->get_data(), PAGE_SIZE);
47         page->is_dirty_ = false;
48     }
49     if (page->get_page_id().page_no != INVALID_PAGE_ID) {
50         page_table_.erase(page->get_page_id());
51     }
52     page_table_[new_page_id] = new_frame_id;
53     page->id_ = new_page_id; // 使用正确的成员名 id_
54     page->reset_memory();
55 }
56
57 /**
58  * @description: 从buffer pool获取需要的页。
59  *
60  * 如果页表中存在page_id (说明该page在缓冲池中), 并且pin_count++。
61  *
62  * 如果页表不存在page_id (说明该page在磁盘中), 则找缓冲池victim page, 将其
63  * 替换为磁盘中读取的page, pin_count置1。
64  * @return {Page*} 若获得了需要的页则将其返回, 否则返回nullptr

```

```

59  * @param {PageId} page_id 需要获取的页的 PageId
60  */
61  Page* BufferPoolManager::fetch_page(PageId page_id) {
62      //Todo:
63      // 1. 从page_table_中搜寻目标页
64      // 1.1 若目标页有被page_table_记录, 则将其所在frame固定(pin), 并返回目标页。
65      // 1.2 否则, 尝试调用find_victim_page获得一个可用的frame, 若失败则返回nullptr
66      // 2. 若获得的可用frame存储的为dirty page, 则须调用update_page将page写回到磁盘
67      // 3. 调用disk_manager_的read_page读取目标页到frame
68      // 4. 固定目标页, 更新pin_count_
69      // 5. 返回目标页
70      std::scoped_lock lock{latch_};
71      if (page_table_.count(page_id)) {
72          frame_id_t frame_id = page_table_[page_id];
73          Page* page = &pages_[frame_id];
74          page->pin_count_++;
75          replacer_->pin(frame_id);
76          return page;
77      }
78      frame_id_t frame_id;
79      if (!find_victim_page(&frame_id)) {
80          return nullptr;
81      }
82      Page* victim_page = &pages_[frame_id];
83      if (victim_page->is_dirty_) {
84          disk_manager_->write_page(victim_page->get_page_id().fd, victim_page->get_page_id()
            .page_no, victim_page->get_data(), PAGE_SIZE);
85      }
86      if (victim_page->get_page_id().page_no != INVALID_PAGE_ID) {
87          page_table_.erase(victim_page->get_page_id());
88      }
89      disk_manager_->read_page(page_id.fd, page_id.page_no, victim_page->get_data(),
        PAGE_SIZE);
90
91      victim_page->id_ = page_id; // 使用正确的成员名 id_
92      victim_page->pin_count_ = 1;
93      victim_page->is_dirty_ = false;
94
95      page_table_[page_id] = frame_id;
96      replacer_->pin(frame_id);
97      return victim_page;
98  }
99
100 /**
101  * @description: 取消固定pin_count>0的在缓冲池中的page
102  * @return {bool} 如果目标页的pin_count<=0则返回false, 否则返回true
103  * @param {PageId} page_id 目标page的page_id
104  * @param {bool} is_dirty 若目标page应该被标记为dirty则为true, 否则为false
105  */
106 bool BufferPoolManager::unpin_page(PageId page_id, bool is_dirty) {
107     // Todo:
108     // 0. lock latch
109     // 1. 尝试在page_table_中搜寻page_id对应的页P
110     // 1.1 P在页表中不存在 return false

```

```

111 // 1.2 P在页表中存在, 获取其pin_count_
112 // 2.1 若pin_count_已经等于0, 则返回false
113 // 2.2 若pin_count_大于0, 则pin_count_自减一
114 // 2.2.1 若自减后等于0, 则调用replacer_的Unpin
115 // 3 根据参数is_dirty, 更改P的is_dirty_
116 std::scoped_lock lock{latch_};
117 if (page_table_.find(page_id) == page_table_.end()) {
118     return false;
119 }
120 frame_id_t frame_id = page_table_[page_id];
121 Page* page = &pages_[frame_id];
122 if (page->pin_count_ <= 0) {
123     return false;
124 }
125 page->pin_count_--;
126 if (page->pin_count_ == 0) {
127     replacer_>unpin(frame_id);
128 }
129 if (is_dirty) {
130     page->is_dirty_ = true;
131 }
132 return true;
133 }
134
135 /**
136  * @description: 将目标页写回磁盘, 不考虑当前页面是否正在被使用
137  * @return {bool} 成功则返回true, 否则返回false(只有page_table_中没有目标页时)
138  * @param {PageId} page_id 目标页的page_id, 不能为INVALID_PAGE_ID
139  */
140 bool BufferPoolManager::flush_page(PageId page_id) {
141     // Todo:
142     // 0. lock latch
143     // 1. 查找页表, 尝试获取目标页P
144     // 1.1 目标页P没有被page_table_记录, 返回false
145     // 2. 无论P是否为脏都将其写回磁盘。
146     // 3. 更新P的is_dirty_
147     std::scoped_lock lock{latch_};
148     if (page_table_.find(page_id) == page_table_.end()) {
149         return false;
150     }
151     frame_id_t frame_id = page_table_[page_id];
152     Page* page = &pages_[frame_id];
153     disk_manager_>write_page(page->get_page_id().fd, page->get_page_id().page_no, page->
        get_data(), PAGE_SIZE);
154     page->is_dirty_ = false;
155     return true;
156 }
157
158 /**
159  * @description: 创建一个新的page, 即从磁盘上移动一个新建的空page到缓冲池某个位置。
160  * @return {Page*} 返回新创建的page, 若创建失败则返回nullptr
161  * @param {PageId*} page_id 当成功创建一个新的page时存储其page_id
162  */
163 Page* BufferPoolManager::new_page(PageId* page_id) {

```

```

164 // 1. 获得一个可用的 frame, 若无法获得则返回 nullptr
165 // 2. 在 fd 对应的文件分配一个新的 page_id
166 // 3. 将 frame 的数据写回磁盘
167 // 4. 固定 frame, 更新 pin_count
168 // 5. 返回获得的 page
169 std::scoped_lock lock{latch_};
170 frame_id_t frame_id;
171 if (!find_victim_page(&frame_id)) {
172     return nullptr;
173 }
174 Page* new_page_frame = &pages_[frame_id];
175 if (new_page_frame->is_dirty_) {
176     disk_manager->write_page(new_page_frame->get_page_id().fd, new_page_frame->
        get_page_id().page_no, new_page_frame->get_data(), PAGE_SIZE);
177 }
178 if (new_page_frame->get_page_id().page_no != INVALID_PAGE_ID) {
179     page_table_.erase(new_page_frame->get_page_id());
180 }
181 page_id_t new_page_no = disk_manager->allocate_page(page_id->fd);
182 page_id->page_no = new_page_no;
183
184 new_page_frame->id_ = *page_id; // 使用正确的成员名 id_
185 new_page_frame->pin_count_ = 1;
186 new_page_frame->is_dirty_ = false;
187 new_page_frame->reset_memory();
188
189 page_table_[*page_id] = frame_id;
190 replacer->pin(frame_id);
191 return new_page_frame;
192 }
193
194 /**
195  * @description: 从 buffer_pool 删除目标页
196  * @return {bool} 如果目标页不存在于 buffer_pool 或者成功被删除则返回 true, 若其存在于
        buffer_pool 但无法删除则返回 false
197  * @param {PageId} page_id 目标页
198  */
199 bool BufferPoolManager::delete_page(PageId page_id) {
200     // 1. 在 page_table_ 中查找目标页, 若不存在返回 true
201     // 2. 若目标页的 pin_count 不为 0, 则返回 false
202     // 3. 将目标页数据写回磁盘, 从页表中删除目标页, 重置其元数据, 将其加入 free_list_,
        返回 true
203     std::scoped_lock lock{latch_};
204     if (page_table_.find(page_id) == page_table_.end()) {
205         return true;
206     }
207     frame_id_t frame_id = page_table_[page_id];
208     Page* page = &pages_[frame_id];
209     if (page->pin_count_ > 0) {
210         return false;
211     }
212     if (page->is_dirty_) {
213         disk_manager->write_page(page->get_page_id().fd, page->get_page_id().page_no,
            page->get_data(), PAGE_SIZE);

```

```

214     }
215     page_table_.erase(page_id);
216     replacer_>pin(frame_id);
217
218     page->id_.page_no = INVALID_PAGE_ID; // 使用正确的成员名 id_
219     page->pin_count_ = 0;
220     page->is_dirty_ = false;
221     page->reset_memory();
222
223     free_list_.push_back(frame_id);
224     return true;
225 }
226
227 /**
228  * @description: 将buffer_pool中的所有页写回到磁盘
229  * @param {int} fd 文件句柄
230  */
231 void BufferPoolManager::flush_all_pages(int fd) {
232     std::scoped_lock lock{latch_};
233     for (auto const& [page_id_val, frame_id_val] : page_table_) {
234         if (page_id_val.fd == fd) {
235             Page* page = &pages_[frame_id_val];
236             if (page->is_dirty_) {
237                 disk_manager_>write_page(page->get_page_id().fd, page->get_page_id().
238                     page_no, page->get_data(), PAGE_SIZE);
239                 page->is_dirty_ = false;
240             }
241         }
242     }
243 }

```

Listing 19: buffer_pool_manager.cpp

A.4 rm_file_handle.cpp

```

1  /* Copyright (c) 2023 Renmin University of China
2  RMDB is licensed under Mulan PSL v2.
3  You can use this software according to the terms and conditions of the Mulan PSL v2.
4  You may obtain a copy of Mulan PSL v2 at:
5      http://license.coscl.org.cn/MulanPSL2
6  THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, WITHOUT WARRANTIES OF ANY KIND,
7  EITHER EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO NON-INFRINGEMENT,
8  MERCHANTABILITY OR FIT FOR A PARTICULAR PURPOSE.
9  See the Mulan PSL v2 for more details. */
10
11 #include "rm_file_handle.h"
12 #include "rm_manager.h"
13
14 /**
15  * @description: 获取当前表中记录号为rid的记录
16  * @param {Rid&} rid 记录号, 指定记录的位置
17  * @param {Context*} context

```

```

18  * @return {unique_ptr<RmRecord>} rid对应的记录对象指针
19  */
20  std::unique_ptr<RmRecord> RmFileHandle::get_record(const Rid& rid, Context* context)
    const {
21      RmPageHandle page_handle = fetch_page_handle(rid.page_no);
22
23      if (!Bitmap::is_set(page_handle.bitmap, rid.slot_no)) {
24          buffer_pool_manager_->unpin_page({fd_, rid.page_no}, false);
25          throw RecordNotFoundError(rid.page_no, rid.slot_no);
26      }
27
28      char* slot = page_handle.get_slot(rid.slot_no);
29      auto record = std::make_unique<RmRecord>(file_hdr_.record_size);
30      memcpy(record->data, slot, file_hdr_.record_size);
31
32      buffer_pool_manager_->unpin_page({fd_, rid.page_no}, false);
33
34      return record;
35  }
36
37  /**
38   * @description: 在当前表中插入一条记录, 不指定插入位置
39   * @param {char*} buf 要插入的记录的数据
40   * @param {Context*} context
41   * @return {Rid} 插入的记录的记录号 (位置)
42   */
43  Rid RmFileHandle::insert_record(char* buf, Context* context) {
44      RmPageHandle page_handle = create_page_handle();
45
46      int slot_no = Bitmap::first_bit(false, page_handle.bitmap, file_hdr_.
        num_records_per_page);
47
48      if (slot_no == file_hdr_.num_records_per_page) {
49          buffer_pool_manager_->unpin_page(page_handle.page->get_page_id(), false);
50          throw InternalError("Page is full, but was expected to have space.");
51      }
52
53      memcpy(page_handle.get_slot(slot_no), buf, file_hdr_.record_size);
54
55      Bitmap::set(page_handle.bitmap, slot_no);
56      page_handle.page_hdr->num_records++;
57
58      if (page_handle.page_hdr->num_records == file_hdr_.num_records_per_page) {
59          file_hdr_.first_free_page_no = page_handle.page_hdr->next_free_page_no;
60      }
61
62      PageId page_id = page_handle.page->get_page_id();
63      buffer_pool_manager_->unpin_page(page_id, true);
64
65      return Rid{page_id.page_no, slot_no};
66  }
67
68
69  /**

```



```

70  * @description: 删除记录文件中记录号为rid的记录
71  * @param {Rid&} rid 要删除的记录的记录号 (位置)
72  * @param {Context*} context
73  */
74  void RmFileHandle::delete_record(const Rid& rid, Context* context) {
75      RmPageHandle page_handle = fetch_page_handle(rid.page_no);
76
77      if (!Bitmap::is_set(page_handle.bitmap, rid.slot_no)) {
78          buffer_pool_manager_>unpin_page({fd_, rid.page_no}, false);
79          throw RecordNotFoundError(rid.page_no, rid.slot_no);
80      }
81
82      bool was_full = (page_handle.page_hdr->num_records == file_hdr_.num_records_per_page)
83          ;
84
85      Bitmap::reset(page_handle.bitmap, rid.slot_no);
86      page_handle.page_hdr->num_records--;
87
88      if (was_full && page_handle.page_hdr->num_records < file_hdr_.num_records_per_page) {
89          release_page_handle(page_handle);
90      }
91
92      buffer_pool_manager_>unpin_page({fd_, rid.page_no}, true);
93  }
94
95  /**
96   * @description: 更新记录文件中记录号为rid的记录
97   * @param {Rid&} rid 要更新的记录的记录号 (位置)
98   * @param {char*} buf 新记录的数据
99   * @param {Context*} context
100  */
101  void RmFileHandle::update_record(const Rid& rid, char* buf, Context* context) {
102      RmPageHandle page_handle = fetch_page_handle(rid.page_no);
103
104      if (!Bitmap::is_set(page_handle.bitmap, rid.slot_no)) {
105          buffer_pool_manager_>unpin_page({fd_, rid.page_no}, false);
106          throw RecordNotFoundError(rid.page_no, rid.slot_no);
107      }
108
109      memcpy(page_handle.get_slot(rid.slot_no), buf, file_hdr_.record_size);
110
111      buffer_pool_manager_>unpin_page({fd_, rid.page_no}, true);
112  }
113
114  /**
115   * @description: 获取指定页面的页面句柄
116   * @param {int} page_no 页面号
117   * @return {RmPageHandle} 指定页面的句柄
118   */
119
120  RmPageHandle RmFileHandle::fetch_page_handle(int page_no) const {
121      Page* page = buffer_pool_manager_>fetch_page({fd_, page_no});
122      if (page == nullptr) {

```

```

123         throw PageNotExistError("", page_no);
124     }
125     return RmPageHandle(&file_hdr_, page);
126 }
127
128 /**
129  * @description: 创建一个新的 page handle
130  * @return {RmPageHandle} 新的 PageHandle
131  */
132 RmPageHandle RmFileHandle::create_new_page_handle() {
133     PageId new_page_id;
134     // #####
135     // ##          修正点 (Correction)          ##
136     // ## 在调用 new_page 之前, 必须指定要为哪个文件创建页面。 ##
137     // ## The fd in new_page_id MUST be initialized before ##
138     // ## calling new_page. ##
139     // #####
140     new_page_id.fd = fd_;
141
142     Page* page = buffer_pool_manager->new_page(&new_page_id);
143     if (page == nullptr) {
144         throw InternalError("Failed to create new page, buffer pool may be full.");
145     }
146     assert(new_page_id.fd == fd_);
147
148     RmPageHandle page_handle(&file_hdr_, page);
149     page_handle.page_hdr->num_records = 0;
150     page_handle.page_hdr->next_free_page_no = RM_NO_PAGE;
151     Bitmap::init(page_handle.bitmap, file_hdr_.bitmap_size);
152
153     file_hdr_.num_pages++;
154
155     return page_handle;
156 }
157
158 /**
159  * @brief 创建或获取一个空闲的 page handle
160  * @return RmPageHandle 返回生成的空闲 page handle
161  */
162 RmPageHandle RmFileHandle::create_page_handle() {
163     if (file_hdr_.first_free_page_no != RM_NO_PAGE) {
164         return fetch_page_handle(file_hdr_.first_free_page_no);
165     } else {
166         return create_new_page_handle();
167     }
168 }
169
170 /**
171  * @description: 当一个页面从没有空闲空间的状态变为有空闲空间状态时, 更新文件头和页头中空
172  *               闲页面相关的元数据
173  */
174 void RmFileHandle::release_page_handle(RmPageHandle& page_handle) {
175     page_handle.page_hdr->next_free_page_no = file_hdr_.first_free_page_no;
176     file_hdr_.first_free_page_no = page_handle.page->get_page_id().page_no;

```

176 }

Listing 20: rm_file_handle.cpp

A.5 rm_scan.cpp

```
1  /* Copyright (c) 2023 Renmin University of China
2  RMDB is licensed under Mulan PSL v2.
3  You can use this software according to the terms and conditions of the Mulan PSL v2.
4  You may obtain a copy of Mulan PSL v2 at:
5      http://license.coscl.org.cn/MulanPSL2
6  THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, WITHOUT WARRANTIES OF ANY KIND,
7  EITHER EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO NON-INFRINGEMENT,
8  MERCHANTABILITY OR FIT FOR A PARTICULAR PURPOSE.
9  See the Mulan PSL v2 for more details. */
10
11 #include "rm_scan.h"
12 #include "rm_file_handle.h"
13
14 /**
15  * @brief 初始化file_handle和rid
16  * @param file_handle
17  */
18 RmScan::RmScan(const RmFileHandle *file_handle) : file_handle_(file_handle) {
19     // 数据页从1开始, 0是文件头页
20     rid_ = {1, -1};
21     next();
22 }
23
24 /**
25  * @brief 找到文件中下一个存放了记录的位置
26  */
27 void RmScan::next() {
28     int current_page = rid_.page_no;
29     int current_slot = rid_.slot_no;
30
31     while(current_page < file_handle_>file_hdr_.num_pages) {
32         // 在当前页内查找下一个有效的记录
33         current_slot = Bitmap::next_bit(true, file_handle_>fetch_page_handle(
34             current_page).bitmap,
35                                     file_handle_>file_hdr_.num_records_per_page,
36                                     current_slot);
37
38         if (current_slot < file_handle_>file_hdr_.num_records_per_page) {
39             // 找到了
40             rid_ = {current_page, current_slot};
41             return;
42         }
43
44         // 当前页扫描完毕, 移动到下一页
45         current_page++;
46         current_slot = -1; // 从下一页的第一个槽位开始
```

```
45     }
46
47     // 如果循环结束仍未找到, 说明已到达文件末尾
48     rid_ = {RM_NO_PAGE, -1};
49 }
50
51 /**
52  * @brief 判断是否到达文件末尾
53  */
54 bool RmScan::is_end() const {
55     return rid_.page_no == RM_NO_PAGE;
56 }
57
58 /**
59  * @brief RmScan内部存放的rid
60  */
61 Rid RmScan::rid() const {
62     return rid_;
63 }
```

Listing 21: rm_scan.cpp